UNIVERSITÀ DEGLI STUDI
DI MILANO

Visione Artificiale

Raffaella Lanzarotti

Where are we?

First part: Image formation and Early vision

- Image formation
  - Geometric Camera Models
  - Color spaces
- Image Processing
  - Punctual and spatial processing
  - Feature Extraction
- Reconstruction
  - Camera calibration
  - Stereo Vision
  - Structure from Motion and RGB-d Cameras
- Optical flow
- **Tracking**

Second part: Machine learning for CV

- Linear Neural Network
- Multi Layer Perceptron
- Convolutional Neural Networks
- Recurrent Neural Networks
- Transformers
- Variational Auto-Encoders
- Generative Adversarial Networks
- Graph Neural Networks
- Self-supervised learning
- Vision Language Models

TRACKING

Tracking

Estimating the trajectory of objects moving in a scene

- Modalities:
  - single/multiple object tracking
  - single/moving/multiple cameras
- Challenges:
  - illumination changes
  - shape changes
  - occlusions
- Currently:
  - very active research field (https://www.votchallenge.net/vot2018/trackers.html)
  - auto tracking camera (https://1beyond.com/autotracker)

Single Object Tracking

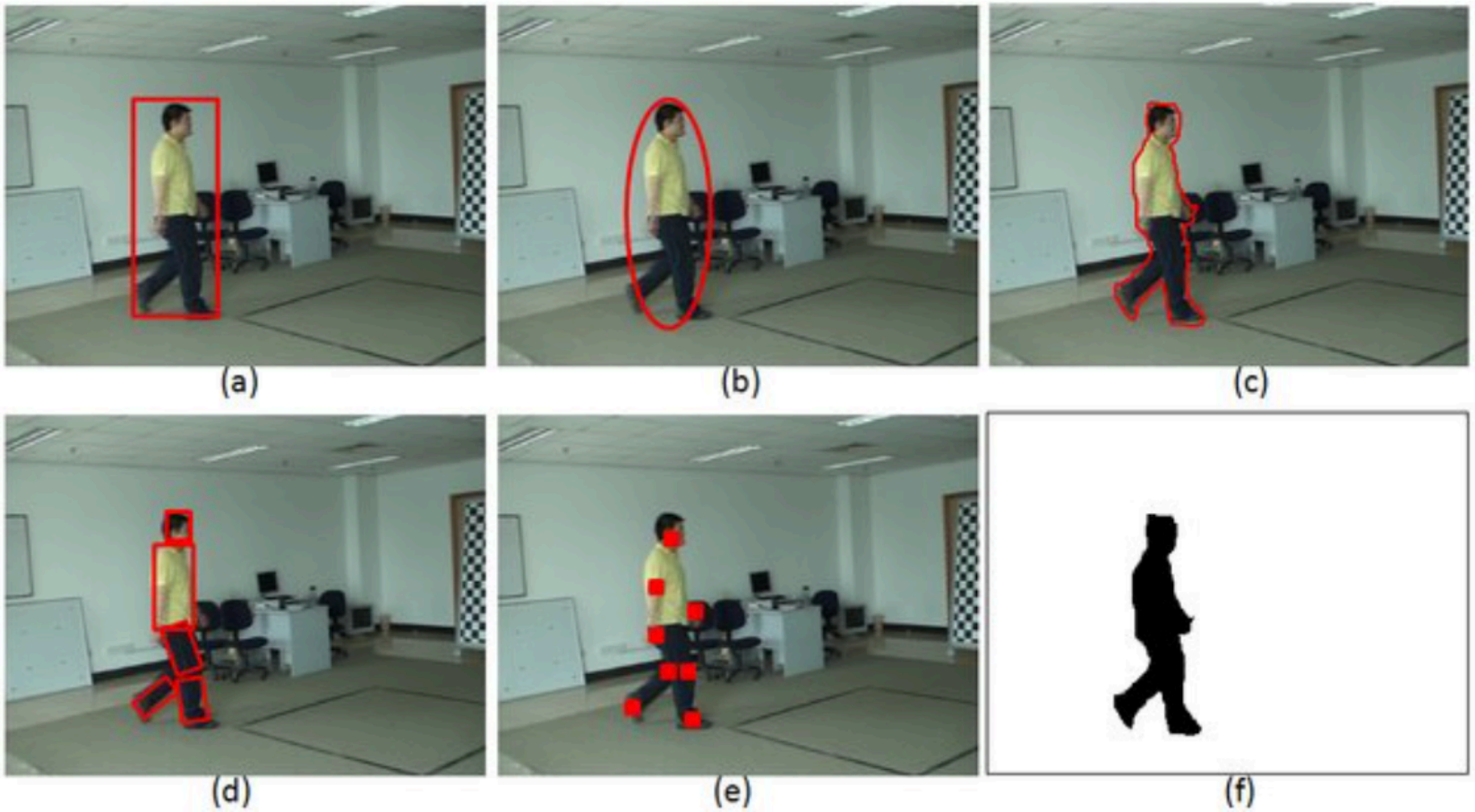
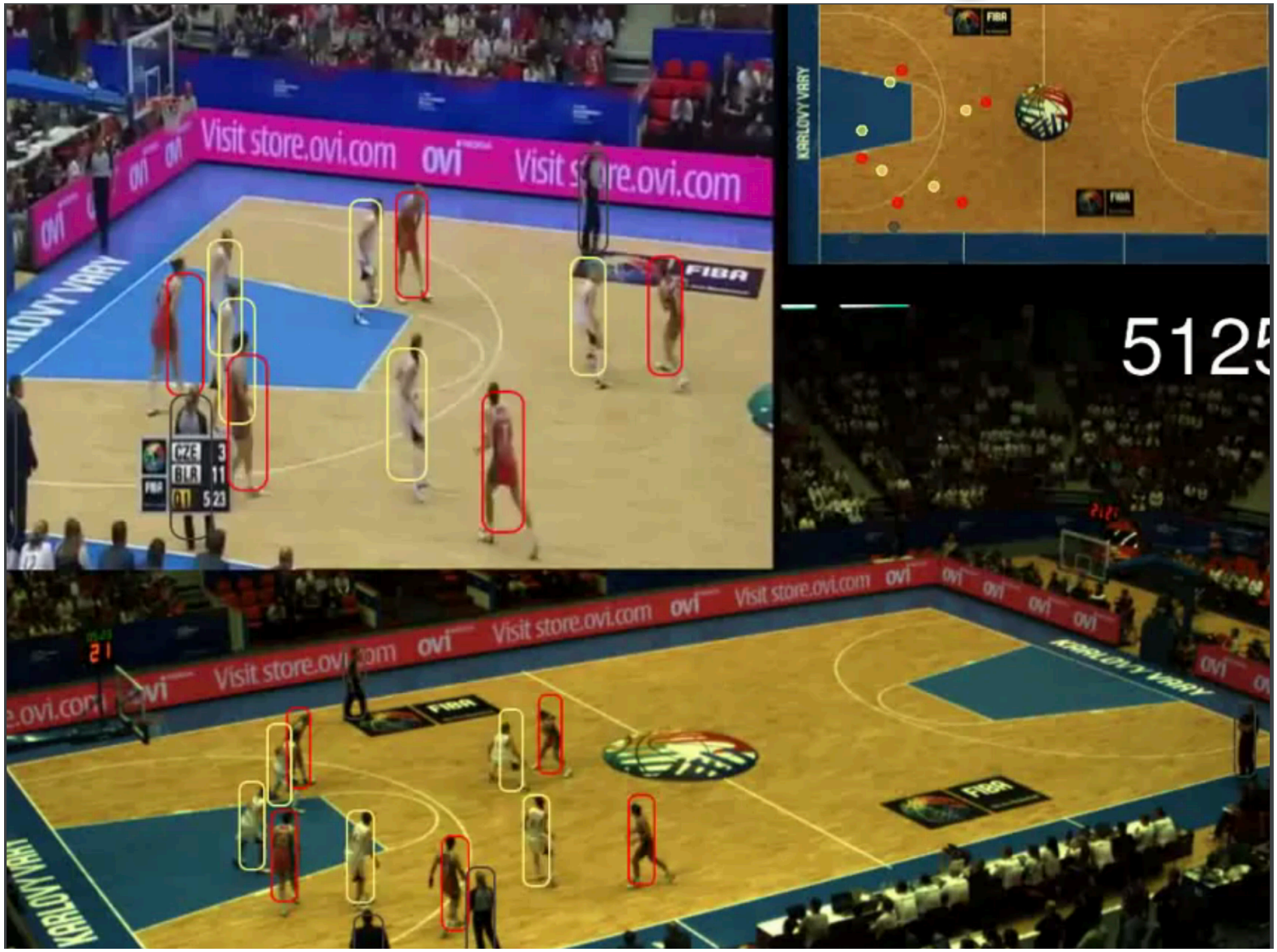


Image by Zhongfei Zhang

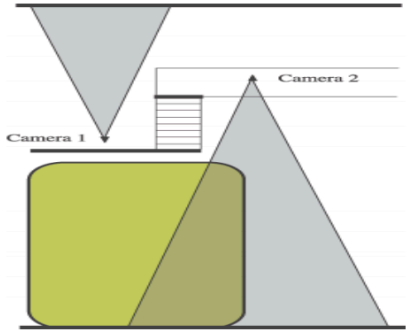
Multiple Object Tracking

Pinter Wollman et al 2011 J Roy Soc Interface

Multiple fixed & Overlapping camera tracking



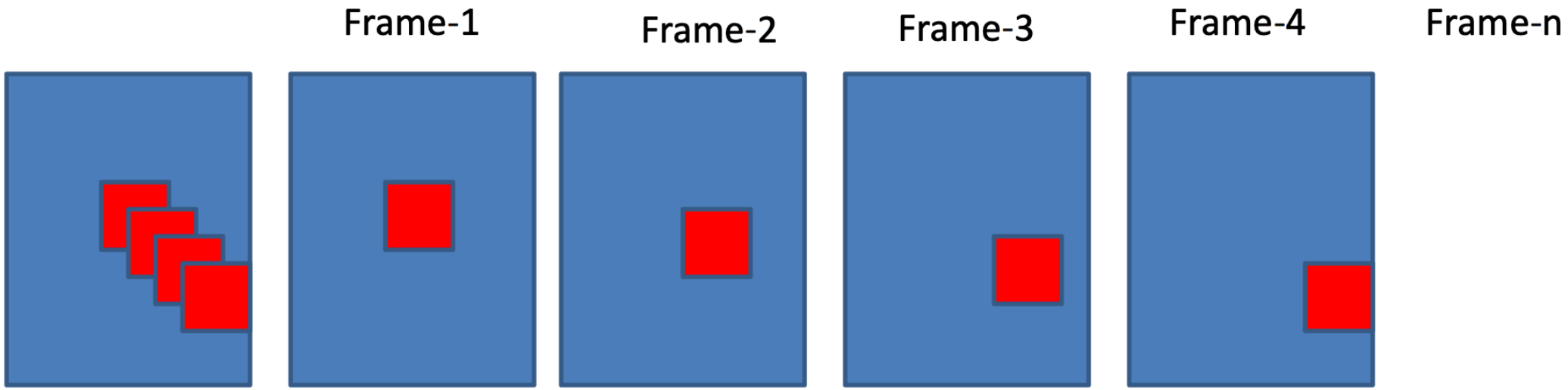
Multiple fixed & Non-overlapping camera tracking



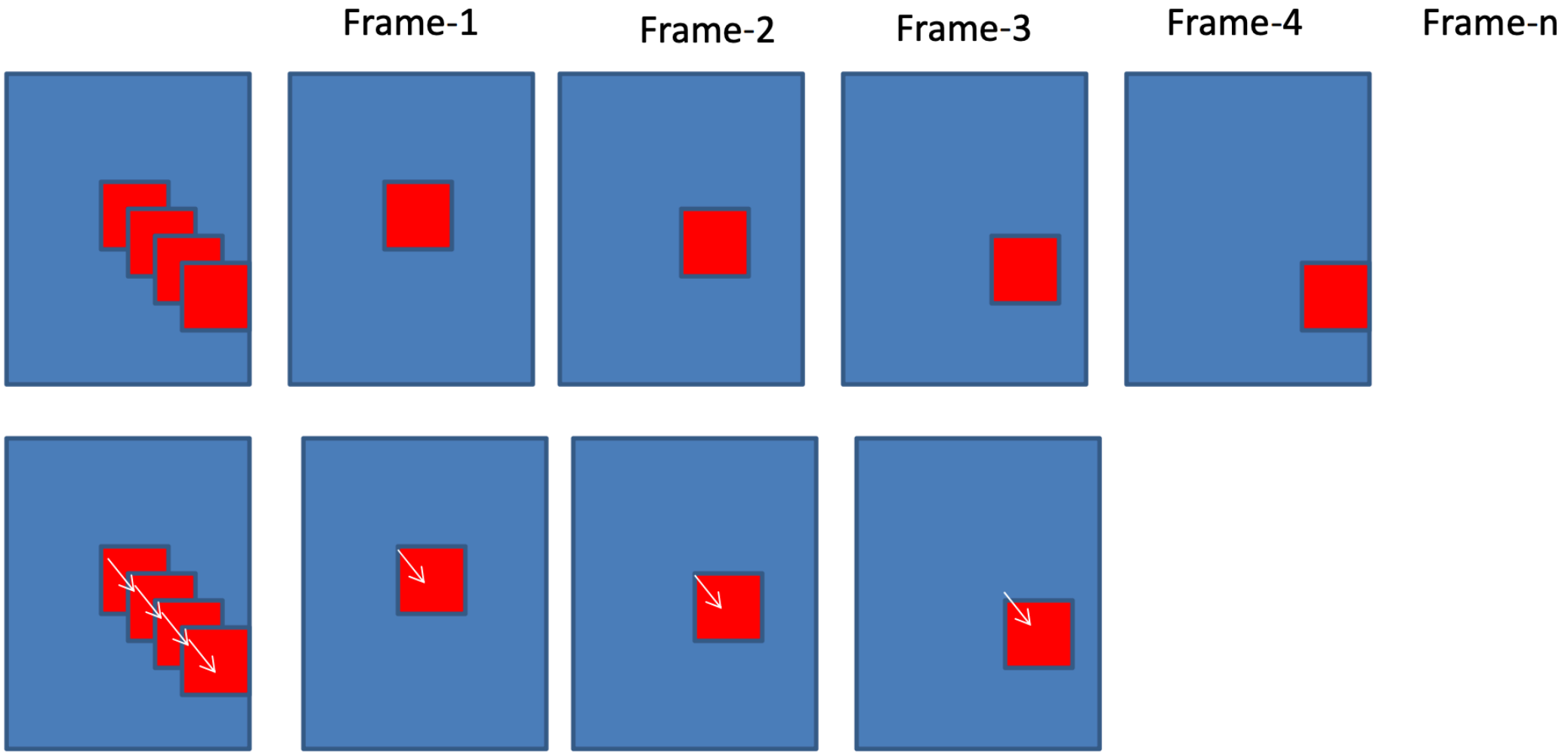
TODAY

FEATURE-BASED MOTION TRACKING

Single object motion



Single object motion

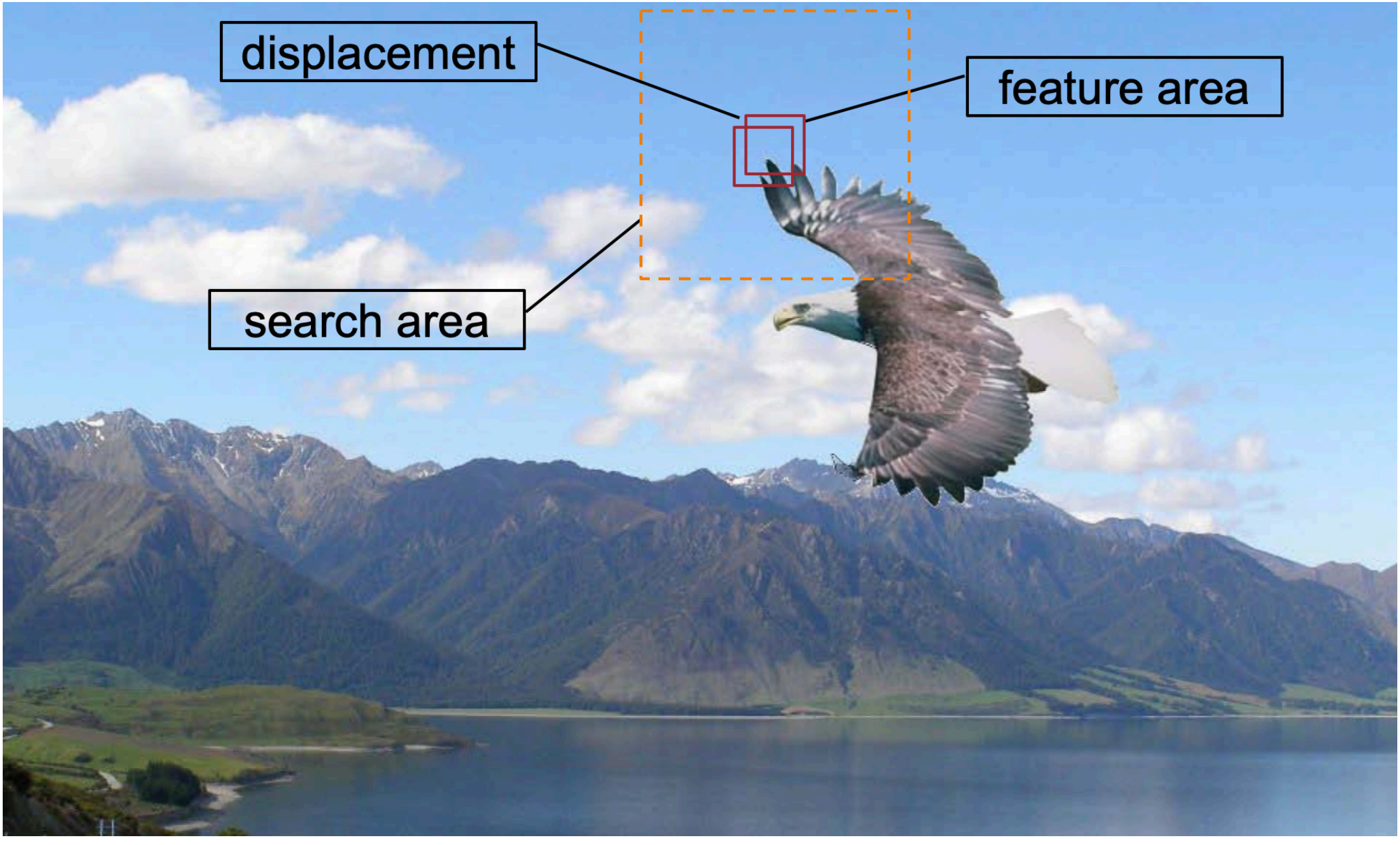


Feature-based motion tracking

- Look for distinct features that change positions

  - the eagle's wings change position
  - the mountains do not change position

KLT (Kanade-Lucas-Tomasi) tracking
Basic idea



KLT (Kanade-Lucas-Tomasi) tracking
Basic idea

1. Look for distinct features in current frame

2. For each feature:

   - Search for matching feature within neighborhood in next frame using Lucas-Kanade OF

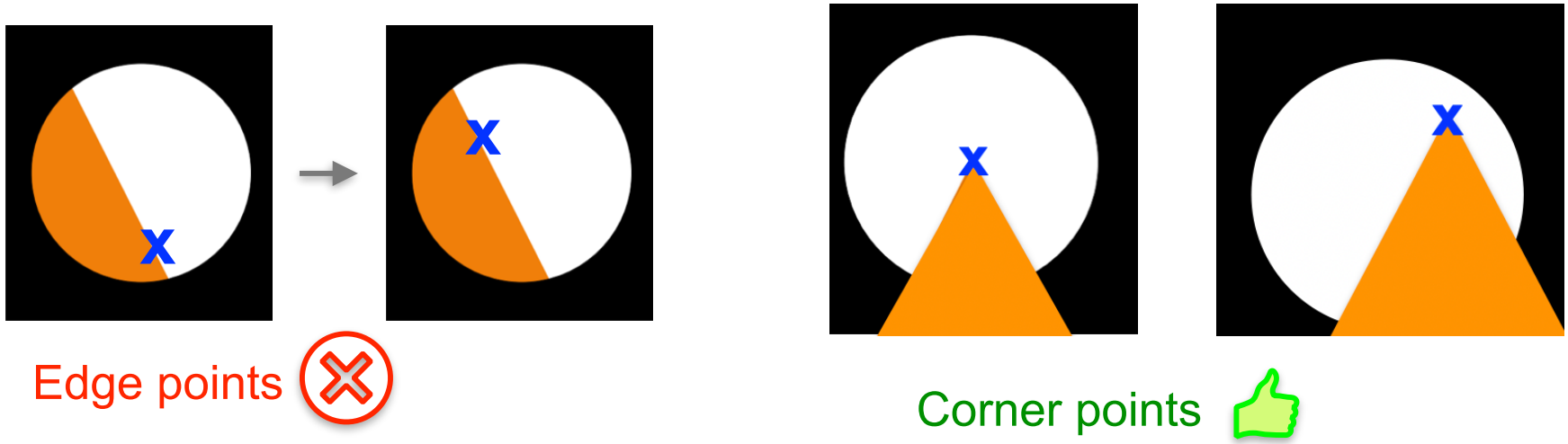
   - Difference in positions → displacement.

   - Velocity = displacement / time difference.

First question: which features?

J. Shi and C. Tomasi. Good Features to Track. CVPR 1994

- Let recall the aperture problem



- Good features to track:
  - Harris corners: $R = \lambda_1\lambda_2 - \alpha(\lambda_1 + \lambda_2)^2$
  - **Shi Tomasi feature:** $R = \min(\lambda_1, \lambda_2)$

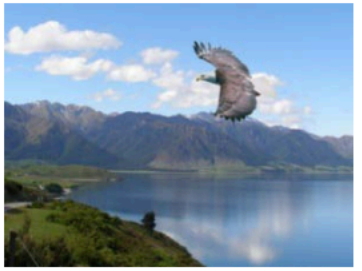
Second question: which neighborhood?

- LK assumes $\mathbf{d}$ small
- this implies the search window W is small too

LKT tracker works well only
for small displacements

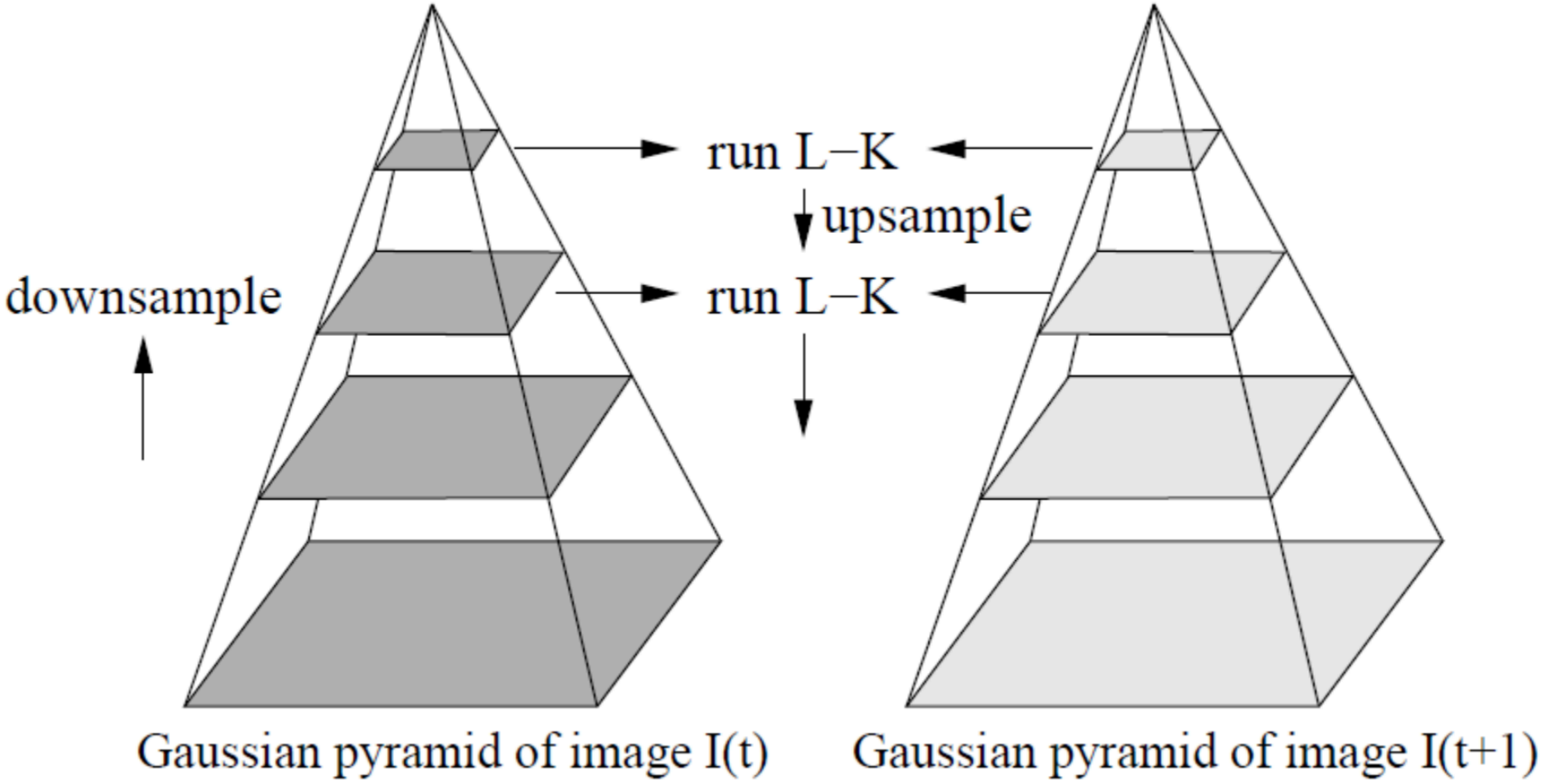
How to handle large displacements?

- Scaling down the image makes the displacements small!



How to handle large displacements?

- LKT using image pyramid



LKT tracker algorithm

- Construct the image pyramids
- detect features (Shi-Tomasi) at the lowest-resolution images
- proceed with a coarse-to-fine search:
  - find displacements at resolution j
  - propagate the result to resolution $j + 1$
  - refine tracking at resolution $j + 1$
  - stop at original resolution
- when long tracks, check appearance of the new patches against the original ones, to detect drifted points

TRACKING WITH DYNAMICS

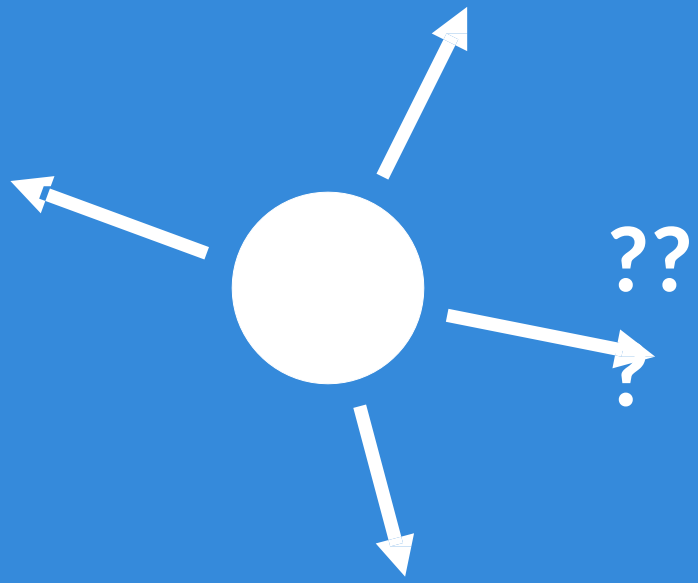
Why do we need to predict

Given an image of a ball,
can you predict where it will go next?

Why do we need to predict

Given an image of a ball,

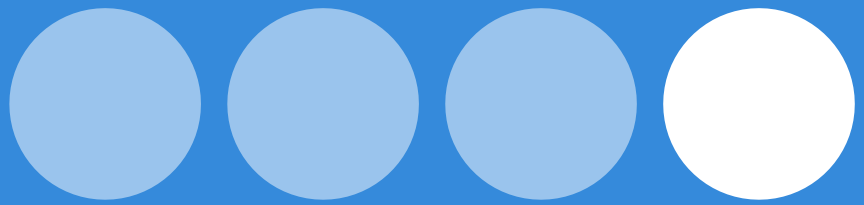
can you predict where it will go next?



Why do we need to predict

Given an image of a ball,

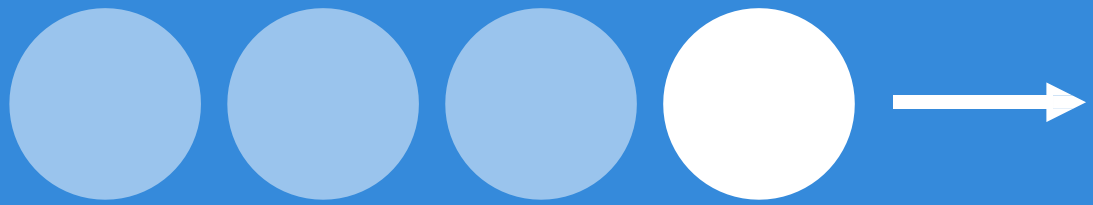
can you predict where it will go next?



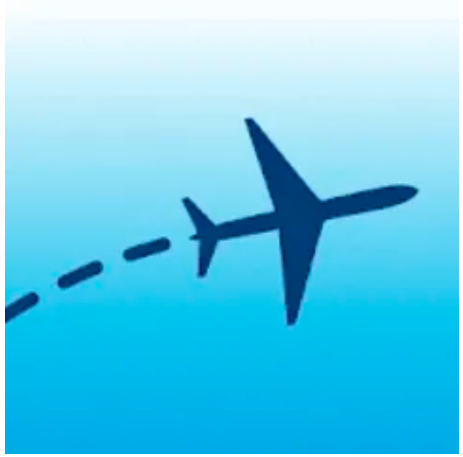
Why do we need to predict

Given an image of a ball,

can you predict where it will go next?



Why do we need to predict



- Suppose we want to estimate the airplane position.
- one way: use sensors such as radar (measurements Z)
- problem: measurements are affected by noise (and occlusions)
- estimation improved using prediction based on dynamics and tracking algorithms

Tracking with dynamics

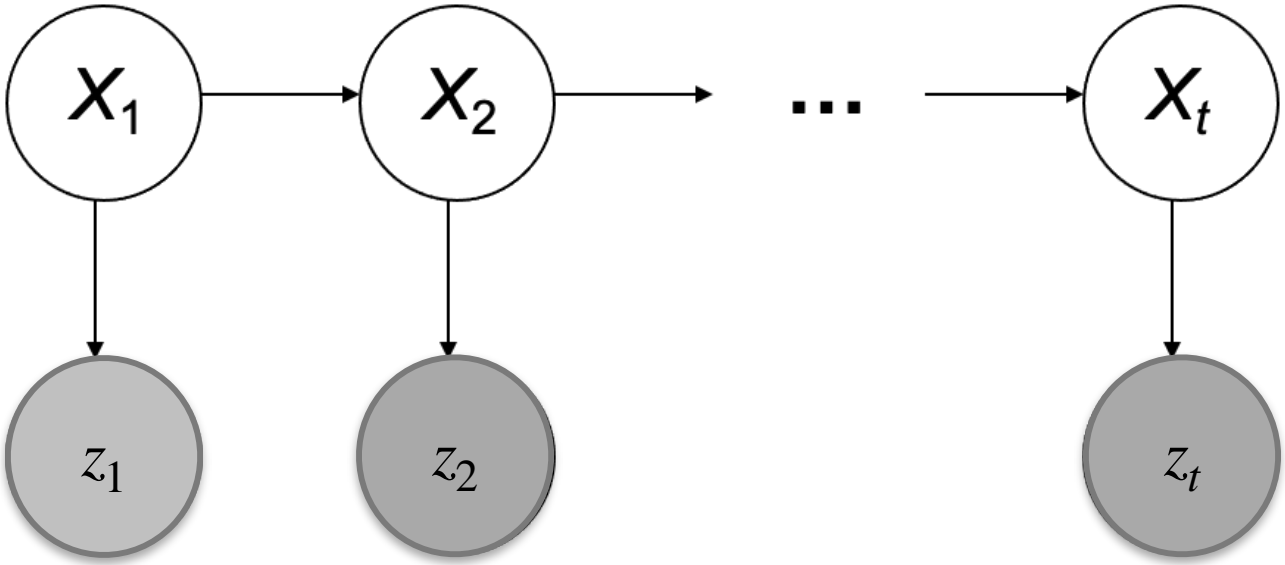
Key idea:
Given a *model of expected motion*, predict where objects will occur in next frame, even before seeing the image

Advantages:
- *Restrict search* for the object

- *Improved estimates since measurement noise is reduced by trajectory smoothness*

General model for tracking

- The moving object of interest is characterized by an **underlying *state* X**
- State X gives rise to ***measurements* or *observations Z***
- At each time i, the state changes to X_i and we get a new observation z_i



observations affected by noise, occlusions, …

Goal: desume the underlying **State**

Kalman filter

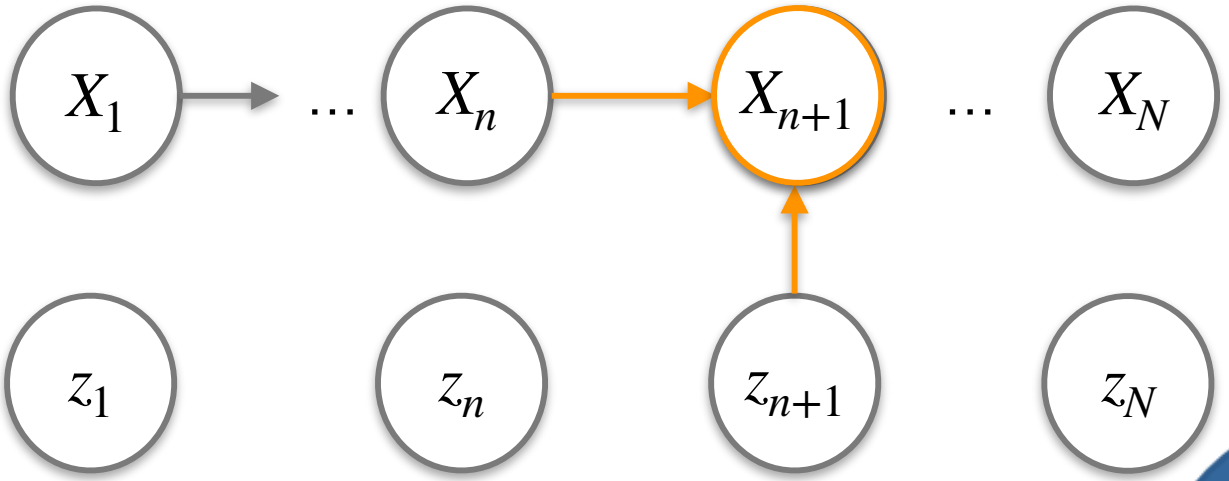
- **"Measure, Update, Predict" algorithm:** produces estimates of hidden variables X based on inaccurate and uncertain measurements Z

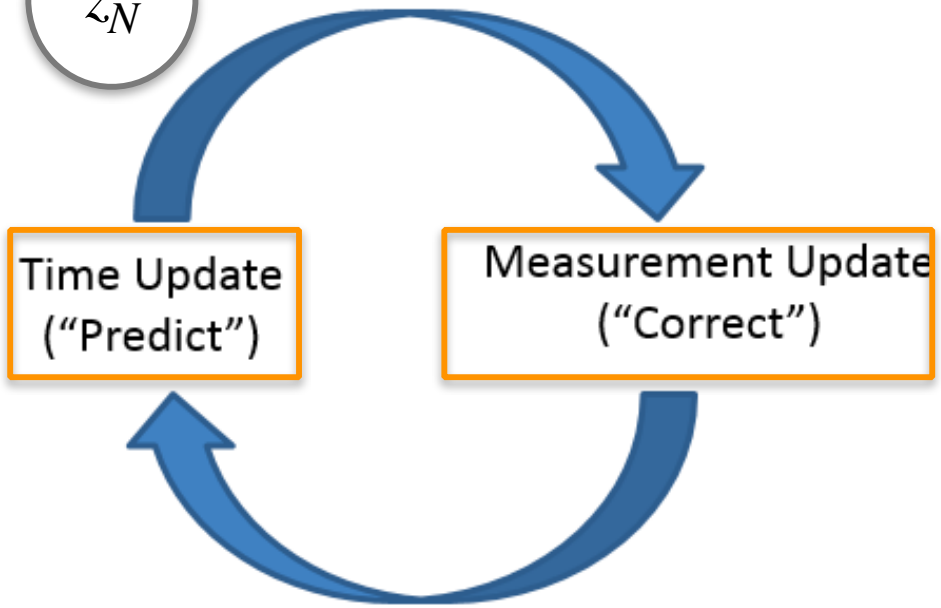
- **Linear dynamic system:** linear prediction of the future system state based on past estimations

- treats measurements, current state estimation, and next state estimation (predictions) as **normally distributed random variables**

Reference:
tutorial

Measure, Update, Predict algorithm

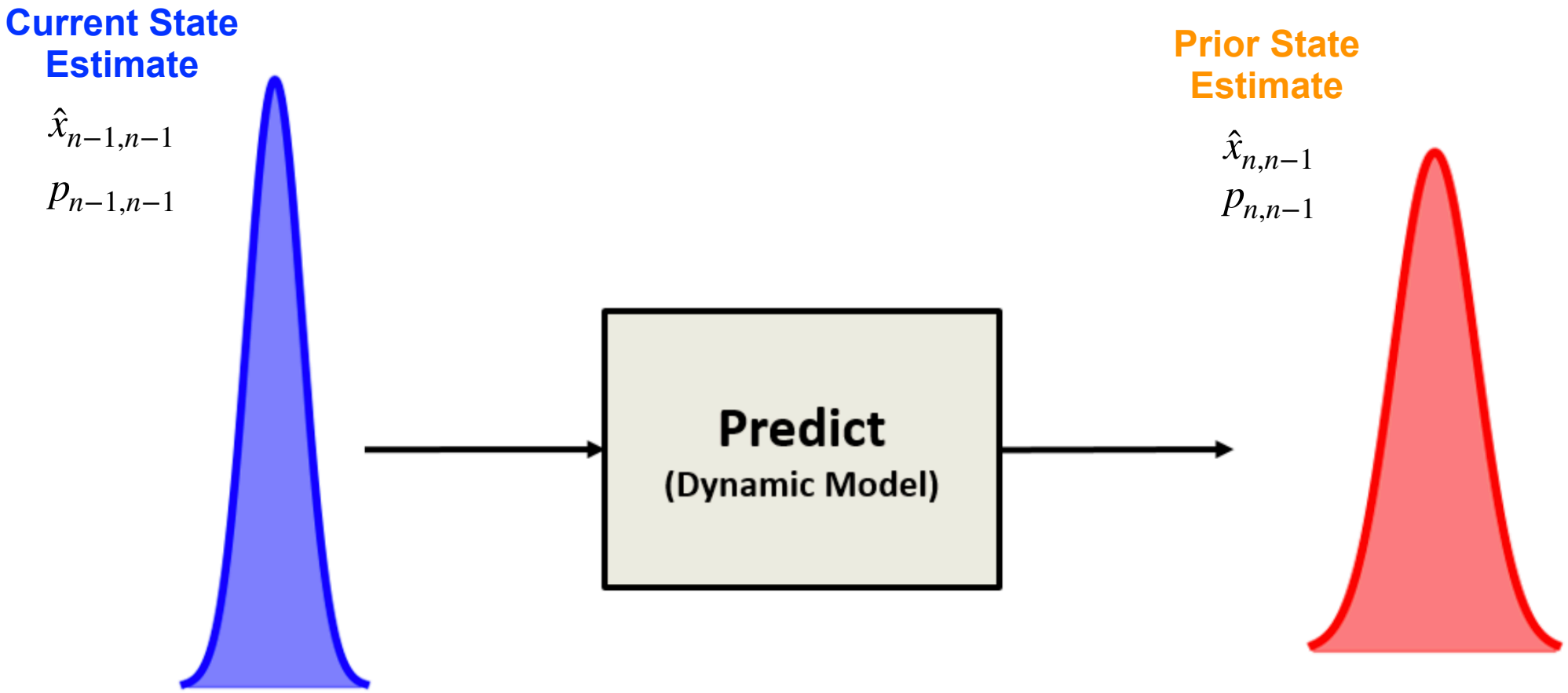




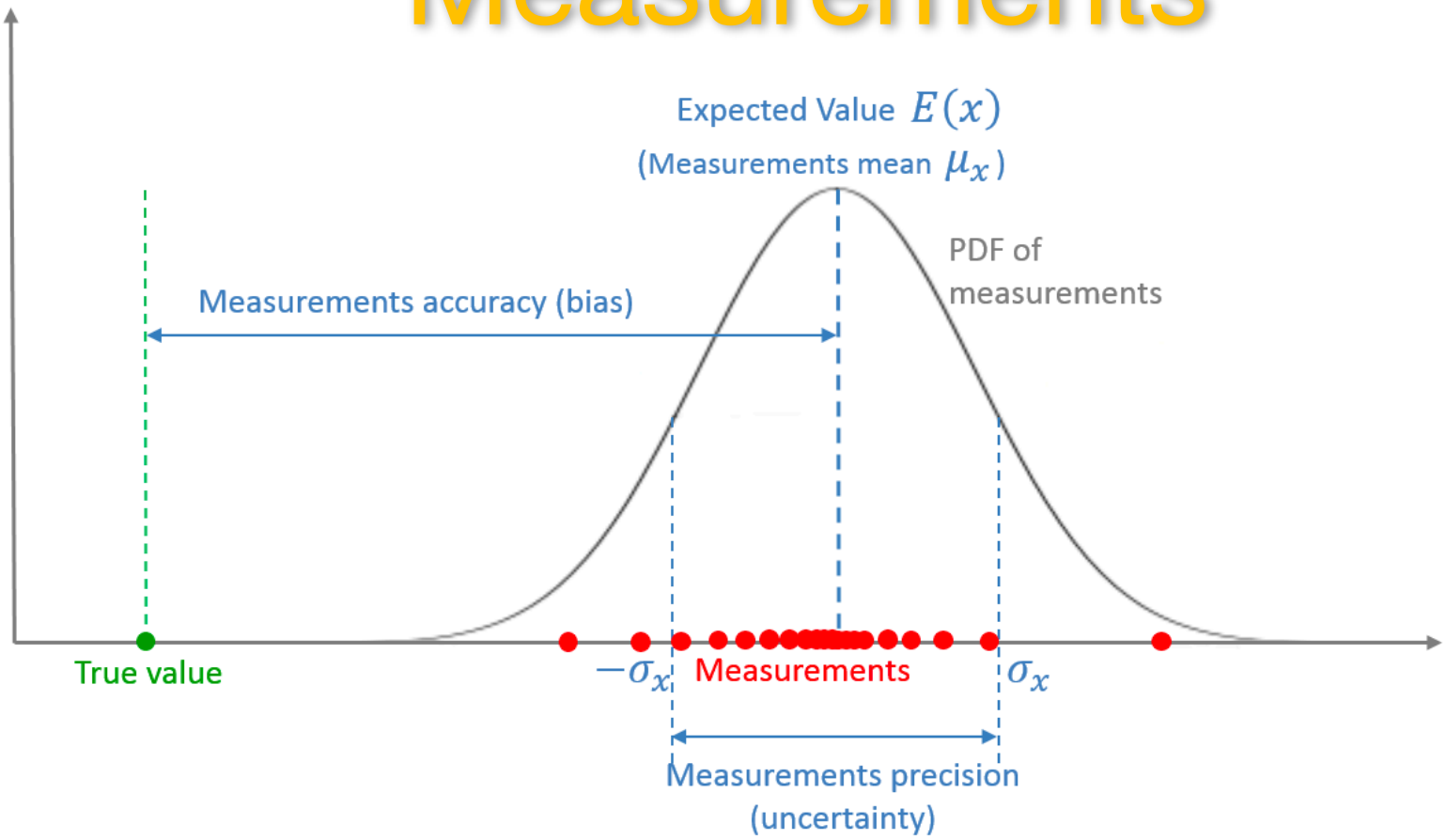
State

- The state is modeled as a normal distribution:
  - x : true state
  - $\hat{x}_{n,n}$: estimate of x at time n
    (having seen the observation z_n)
  - $\hat{x}_{n,n-1}$: prior prediction of x at time n
    (NOT having seen the observation z_n)
  - $p_{n,n}$: estimate uncertainty at time n

Prediction step



Measurements

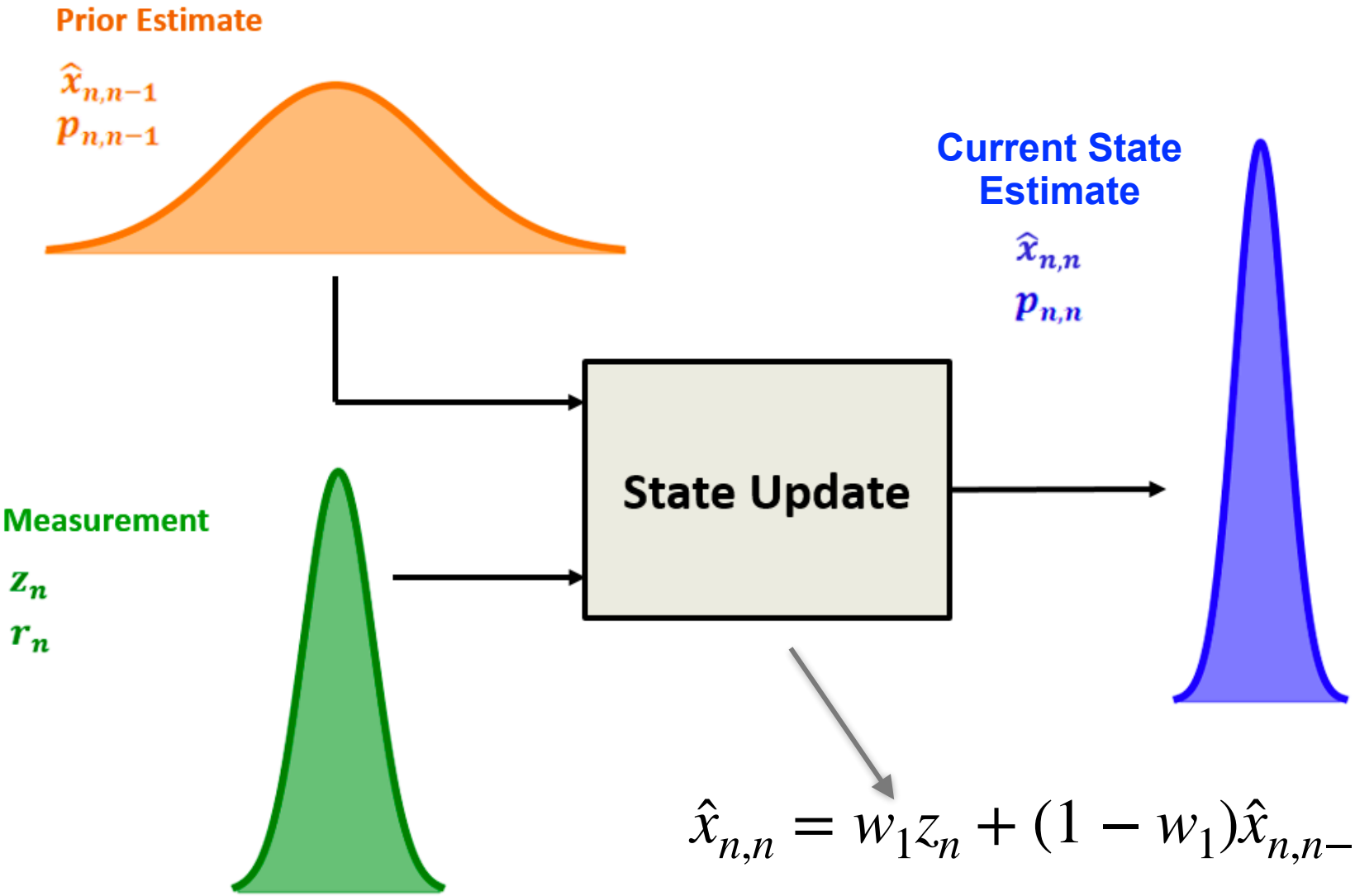


- A measurement is a **random variable**, described by the **Prob. Density Function (PDF)**
- The mean of the measurements is the **Expected Value** of the random variable
- The offset between the mean of the measurements and the true value is the **measurements accuracy**, or **bias**
- The dispersion (variance) of the distribution is the **measurement uncertainty**

Measurements

- The measurement is modeled as a normal distribution:
  - z_n : n-th measurement
  - r_n : measurement uncertainty

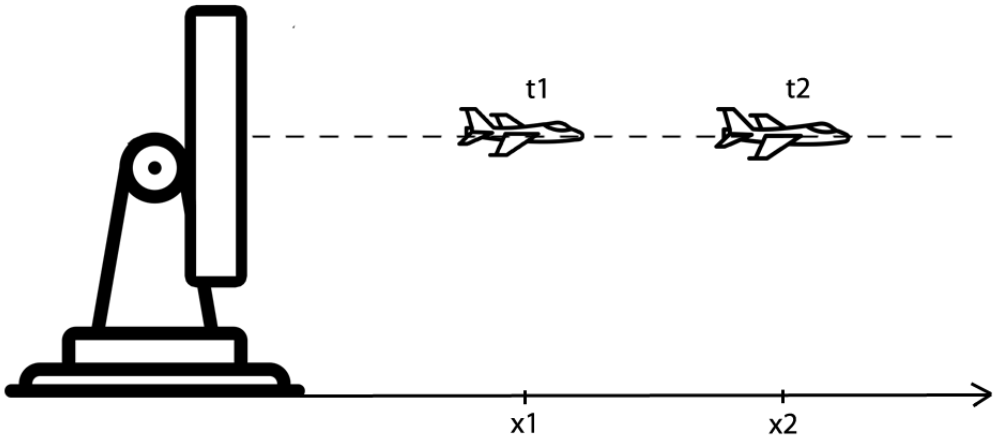
State Update



$$\hat{x}_{n,n} = w_1 z_n + (1 - w_1)\hat{x}_{n,n-1}$$

$$p_{n,n} = (1 - w_1)p_{n,n-1}$$

Example: Tracking 1D with constant velocity

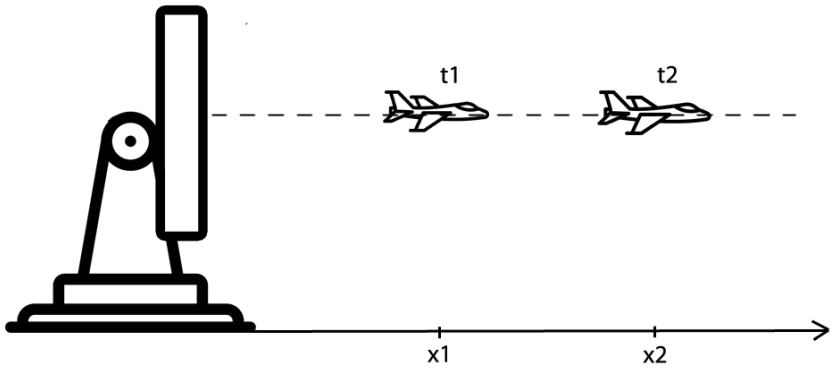


[aircraft moving horizontally]

- system dynamic model for constant velocity motion:

$$x_{t+1} = x_t + \delta t \cdot v_{t+1}$$
$$v_{t+1} = v_t$$

Prediction step



- Prediction equations exploit the system dynamic model. In this case:

$$x_{n+1,n} = x_{n,n} + \Delta t \dot{x}_{n,n}$$

$$\dot{x}_{n+1,n} = \dot{x}_{n,n}$$

where x_n is the position at time n, and $\dot{x} = \dfrac{dx}{dt}$ is the velocity

Estimate of uncertainty

- p^x: position estimate uncertainty
- p^v: velocity estimate uncertainty

Applying the rule:

$$V(x) = \Delta t^2 V(v)$$

or

$$\sigma_x^2 = \Delta t^2 \sigma_v^2$$

Where:

x	is the displacement of the body
v	is the velocity of the body
$\Delta(t)$	is the time interval

- the estimation uncertainty extrapolation is:
  - $p^x_{n+1,n} = p^x_{n,n} + \Delta t^2 p^v_{n,n} + q_n$
  - $p^v_{n+1,n} = p^v_{n,n} + q_n$

q_n : Process Noise
Variance: $N(0,q_n)$

The expectation of the body position variance (proof)

$$V(x) = \sigma_x^2 = E(x^2) - \mu_x^2 =$$

$$E((v\Delta t)^2) - (\mu_v \Delta t)^2 =$$

Express the body position variance in terms of time and velocity $x = \Delta t v$

$$E(v^2 \Delta t^2) - \mu_v^2 \Delta t^2 =$$

$$\Delta t^2 E(v^2) - \mu_v^2 \Delta t^2 =$$

Apply the rule: $E(ax) = aE(x)$

$$\Delta t^2 (E(v^2) - \mu_v^2) =$$

Apply the rule: $V(x) = E(x^2) - \mu_x^2$

$$\Delta t^2 V(v)$$

Correction: State update step

Given the prediction and the measurement,
make a weighted combination of them using w_1 and w_2

- $\hat{x}_{n,n} = w_1 z_n + w_2 \hat{x}_{n,n-1}, \quad w_1 + w_2 = 1$

$$\hat{x}_{n,n} = w_1 z_n + (1 - w_1)\hat{x}_{n,n-1}$$

- $p_{n,n} = w_1^2 r_n + (1 - w_1)^2 p_{n,n-1}$

Kalman Filter: optimal filter

- it combines the prior state estimate with the measurement in a way that minimizes the uncertainty of the current state estimate.
- GOAL: $\text{argmin}_{w_1}\{p_{n,n}\} \rightarrow \frac{d}{dw_1} p_{n,n} = 0$

$$w_1 = \frac{p_{n,n-1}}{p_{n,n-1} + r_n}$$

Kalman Gain

K_n

derived on Tablet

Correction: State update step (cont.)

$$\hat{x}_{n,n} = w_1 z_n + (1 - w_1)\hat{x}_{n,n-1} =$$

$$= \hat{x}_{n,n-1} + w_1(z_n - \hat{x}_{n,n-1}) =$$

$$= \hat{x}_{n,n-1} + K_n(z_n - \hat{x}_{n,n-1})$$

PRIOR

INNOVATION

Kalman Gain

Correction: State update step (cont.)

$$\hat{x}_{n,n} = \hat{x}_{n,n-1} + K_n \left(z_n - \hat{x}_{n,n-1} \right) = \left(1 - K_n \right) \hat{x}_{n,n-1} + K_n z_n$$

- K_n : measurement weight
- $(1 - K_n)$: weight of the current state estimate

INTERPRETATION (remember : $K_n = \dfrac{U_{est}}{U_{est} + U_{meas}} = \dfrac{p_{n,n-1}}{p_{n,n-1} + r_n}$)

- $K_n \approx 0$ when high r_n, small weight to the measurement
- $K_n \approx 1$ when r_n is low and $p_{n,n-1}$ is high, high weight to the measurement

Correction: State update step (cont.)

- Covariance Update Equation:

$$p_{n,n} = \left(1 - K_n\right) p_{n,n-1}$$

- the estimate uncertainty is constantly getting smaller but:
  - it is slow when K_n is low (high measurement uncertainty)
  - it is fast when K_n is high (low measurement uncertainty)

derived on Tablet

Kalman Gain Intuition

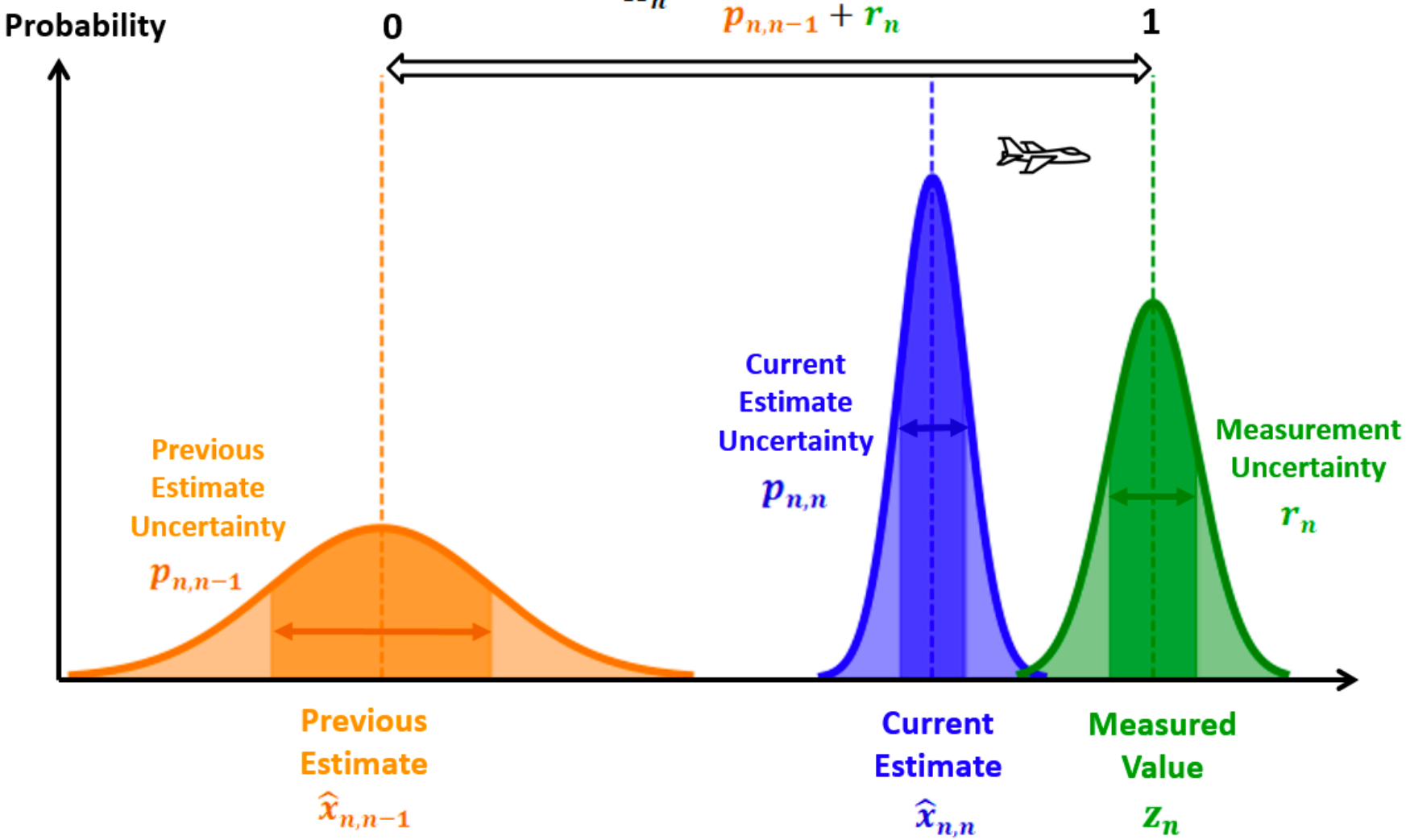
$$\hat{x}_{n,n} = \hat{x}_{n,n-1} + K_n\left(z_n - \hat{x}_{n,n-1}\right)$$

$$p_{n,n} = (1 - K_n)p_{n,n-1}$$

HIGH KALMAN GAIN

due to low measurement uncertainty r_n

$$K_n = \frac{p_{n,n-1}}{p_{n,n-1} + r_n}$$



Kalman Gain Intuition

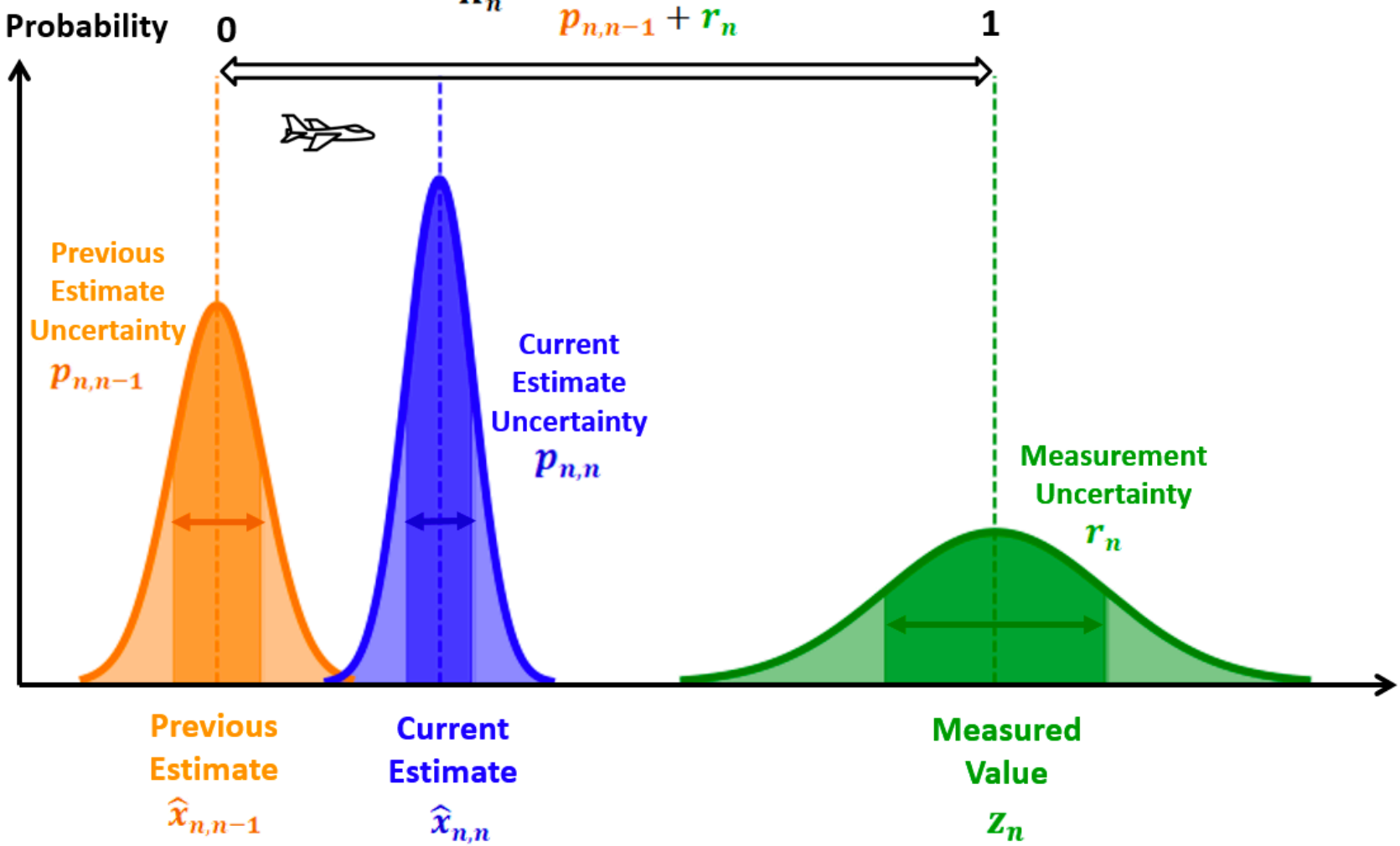
LOW KALMAN GAIN

due to high measurement uncertainty r_n

$$\hat{x}_{n,n} = \hat{x}_{n,n-1} + K_n\left(z_n - \hat{x}_{n,n-1}\right)$$

$$p_{n,n} = (1 - K_n)p_{n,n-1}$$

$$K_n = \frac{p_{n,n-1}}{p_{n,n-1} + r_n}$$



Equations of Karman filter (for 1D velocity constant motion)

Equation	Equation Name
$\hat{x}_{n,n} = \hat{x}_{n,n-1} + K_n \left(z_n - \hat{x}_{n,n-1}\right)$	State Update
$\hat{x}_{n+1,n} = \hat{x}_{n,n} + \Delta t \hat{\dot{x}}_{n,n}$ $\hat{\dot{x}}_{n+1,n} = \hat{\dot{x}}_{n,n}$ (For constant velocity dynamics)	State Prediction
$K_n = \frac{p_{n,n-1}}{p_{n,n-1}+r_n}$	Kalman Gain
$p_{n,n} = (1 - K_n)\, p_{n,n-1}$	Covariance Update
$p^x_{n+1,n} = p^x_{n,n} + \Delta t^2 p^v_{n,n} + q_n$ $p^v_{n+1,n} = p^v_{n,n} + q_n$	Covariance Prediction

Kalman Filter Scheme

PREDICTION:

(e.g. for constant velocity dynamics)

- State Prediction

$$\hat{x}_{n+1,n} = \hat{x}_{n,n} + \Delta t \hat{\dot{x}}_{n,n}$$
$$\hat{\dot{x}}_{n+1,n} = \hat{\dot{x}}_{n,n}$$

- Covariance prediction

$$p^x_{n+1,n} = p^x_{n,n} + \Delta t^2 p^v_{n,n} + q_n$$
$$p^v_{n+1,n} = p^v_{n,n} + q_n$$

CORRECTION:

- Kalman gain

$$K_n = \frac{p_{n,n-1}}{p_{n,n-1} + r_n}$$

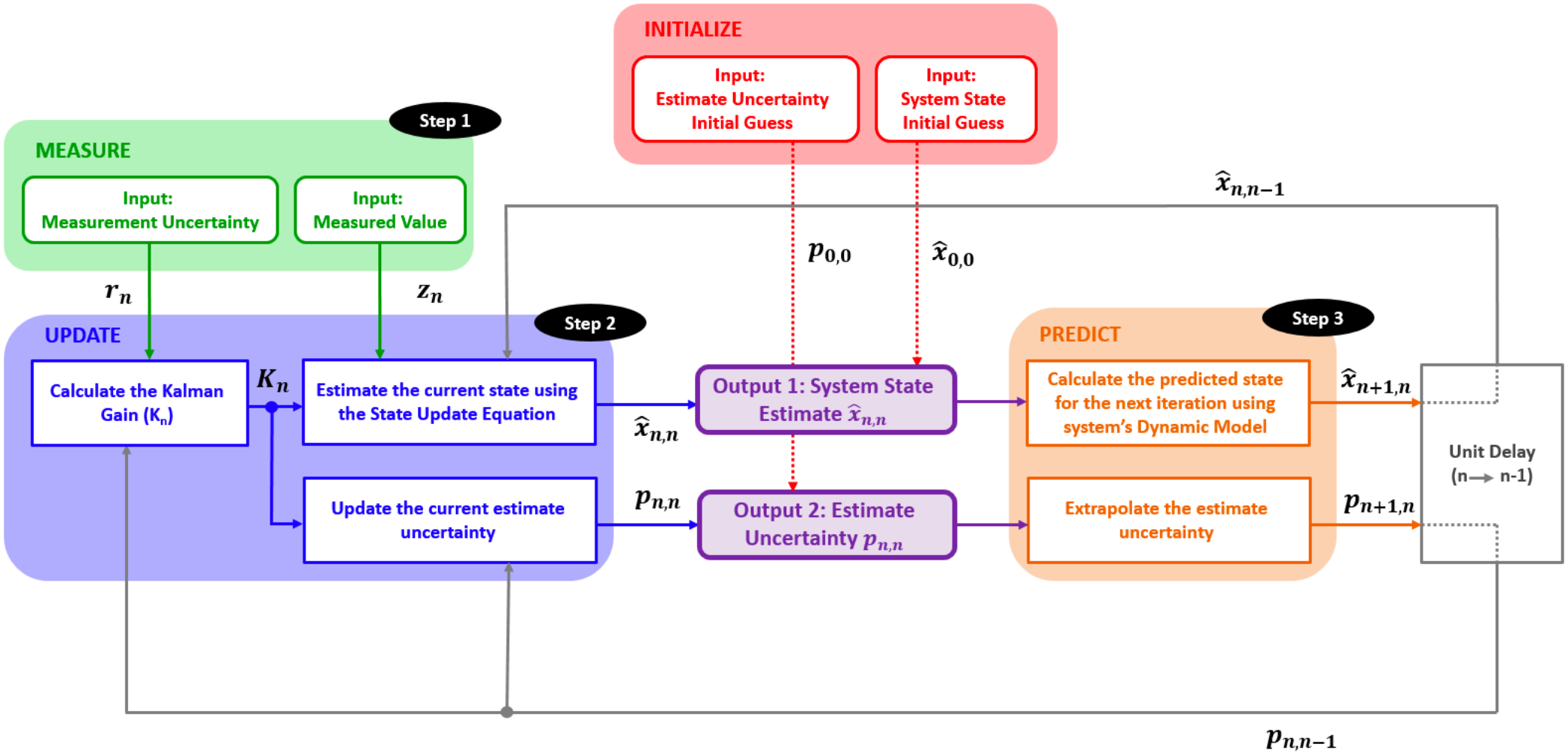
- State update

$$\hat{x}_{n,n} = \hat{x}_{n,n-1} + K_n(z_n - \hat{x}_{n,n-1})$$

- Covariance update

$$p_{n,n} = (1 - K_n)p_{n,n-1}$$

Kalman filter scheme



Reference

Tutorial: https://www.kalmanfilter.net/default.aspx

Lab time

- `m02_objDet_Tracking.ipynb`
- `m03_kalman_filter.ipynb`